



Hire Smarter Not Harder

A data-driven approach to optimizing recruitment efficiency and predicting offer acceptance



Meet the Founding Team



Faisal Khoirudin

Project Manager



Irsan Maulana
Yusuf

Data Analyst



Achmad Kamil

Data Scientist



Taufiq Jonel
Tandra

Data Engineer

We're hiring slower, paying more, and losing candidates

What's changing

- Hybrid work
- Global talent pools
- Skill-based hiring

What it causes

More complex sourcing and candidate evaluation

Key challenges

- Longer time-to-hire
- Rising cost-per-hire
- Declining offer acceptance

Source: [LinkedIn](#)

What does our data tell us?

Avg. time-to-hire

47.2 days

 11.19 days above benchmark
Benchmark: [LinkedIn](#)

Avg. cost-per-hire

\$5,215

 \$465 above benchmark
Benchmark: [SHRM](#)

Avg. offer acceptance

65%

 15% below benchmark
Benchmark: [KPI Depot](#)

The right fit, the right yes

Transforming recruitment from reactive and intuition-driven to data-driven — by predicting which candidates are most likely to accept an offer before hiring decisions are made.

Goal

Develop a decision-support system that predicts offer acceptance and enables more efficient, cost-effective hiring.

Objectives

- 1 Diagnose recruitment inefficiencies using historical hiring data
- 2 Identify key drivers of offer acceptance
- 3 Build a binary classification model to predict acceptance and support decision-making

Success Metrics

$\geq 80\%$ AUC-ROC and F1-score for offer acceptance prediction

Model performance threshold

$\geq 80\%$ Target offer acceptance rate

Up from current 65%

Dataset overview

	num_applicants	time_to_hire_days	cost_per_hire	offer_acceptance_rate
count	5,000	5,000	5,000	5,000
mean	155.62	47.19	5,214.83	0.651
std	84.16	23.87	2,731.00	0.202
min	10	7	507.16	0.300
25%	83	26	2,820.60	0.480
50%	157	47	5,218.29	0.630
75%	229	67	7,611.41	0.830
max	299	89	9,998.91	1.000

Data types

Identifier

- recruitment_id

Categorical

- department
- job_title
- source

Numerical

- num_applicants
- time_to_hire_days
- cost_per_hire
- offer_acceptance_rate

✓ Data quality

0 missing values
duplicates
outliers

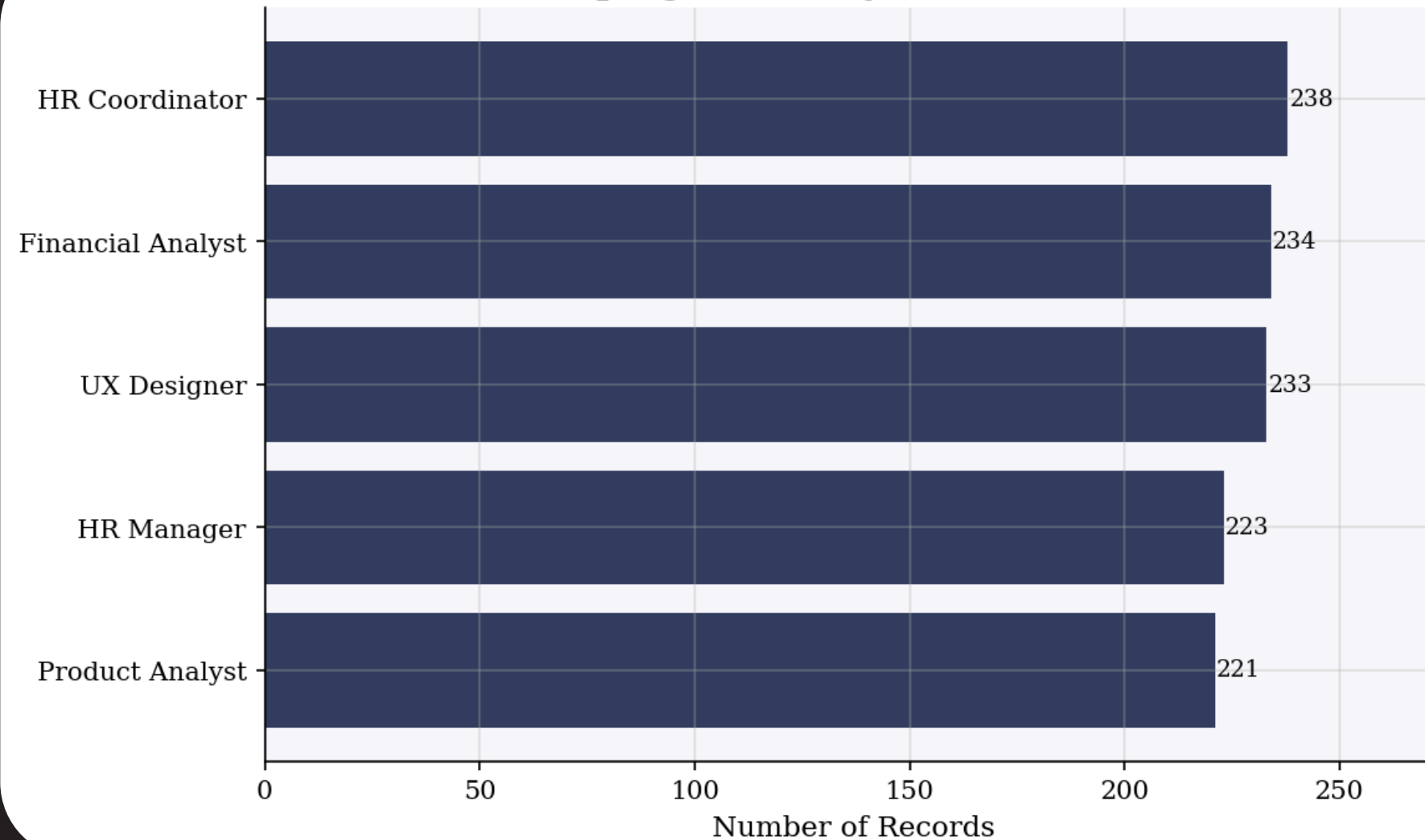
Target feature — Offer Acceptance Rate



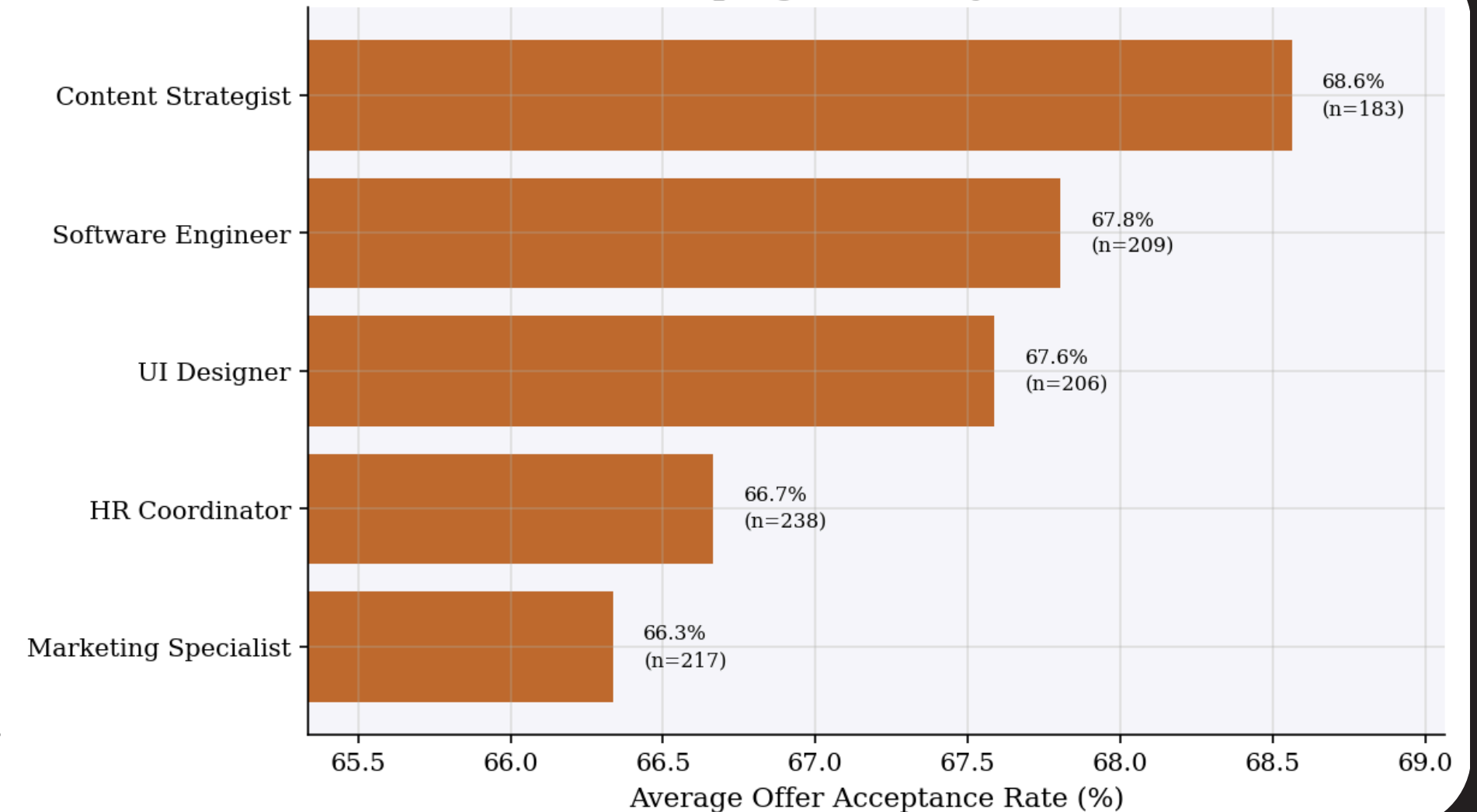
- Low OAR (0) — acceptance rate < 70%
 - High OAR (1) — acceptance rate ≥ 70%
- Threshold based on KPI Depot benchmark

More applicants, not better outcomes

Top 5 Job Title by Record Count

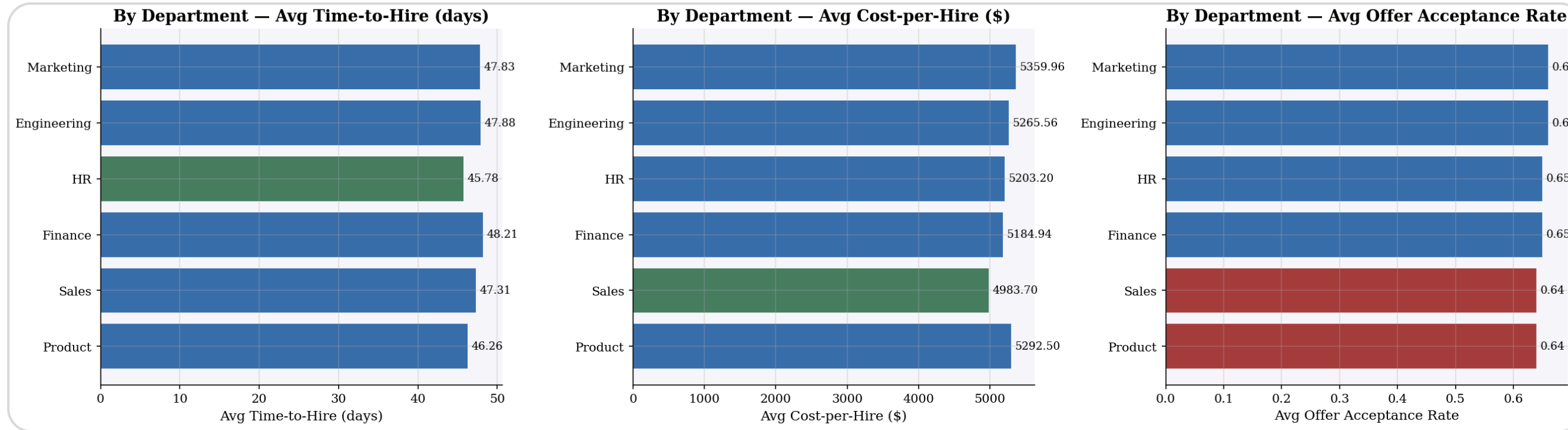


Top 5 Job Title by OAR



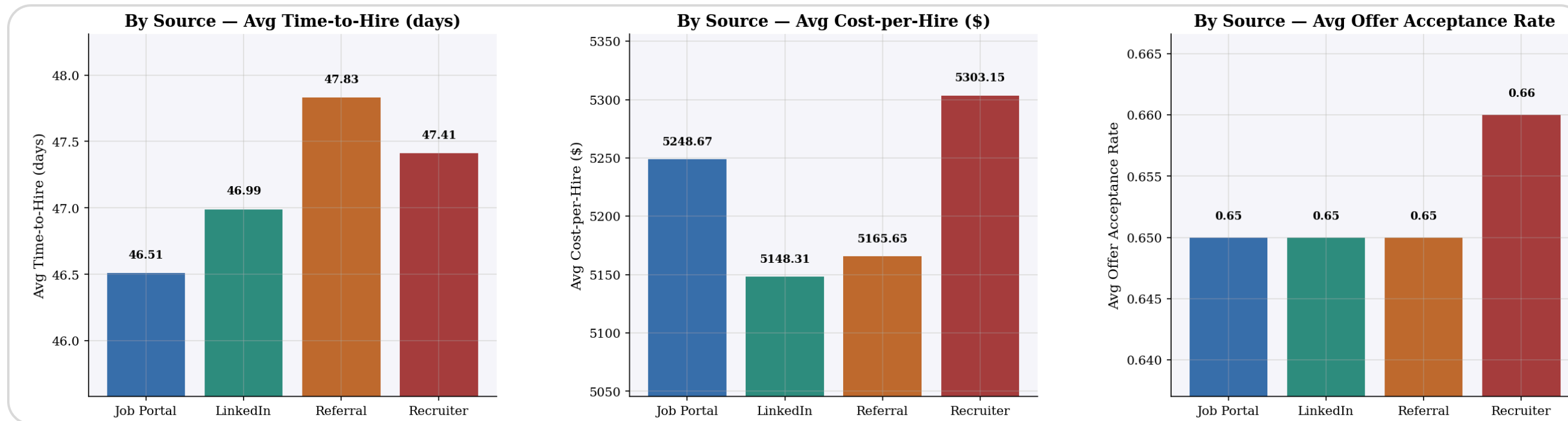
A higher number of applicants does not lead to a higher Offer Acceptance Rate. Roles with large candidate volumes — such as HR Coordinator (238 records) — achieve only moderate acceptance rates (66.7%), while roles like Content Strategist (183 records) top the OAR chart at 68.6%. This indicates that **increasing applicant quantity alone creates additional recruitment cost and longer hiring cycles without improving outcomes.**

Where you hire matters more than who hires



Departments perform equally

All departments show nearly identical TTH, CPH, and OAR — suggesting inefficiencies are **systemic, not department-specific.**

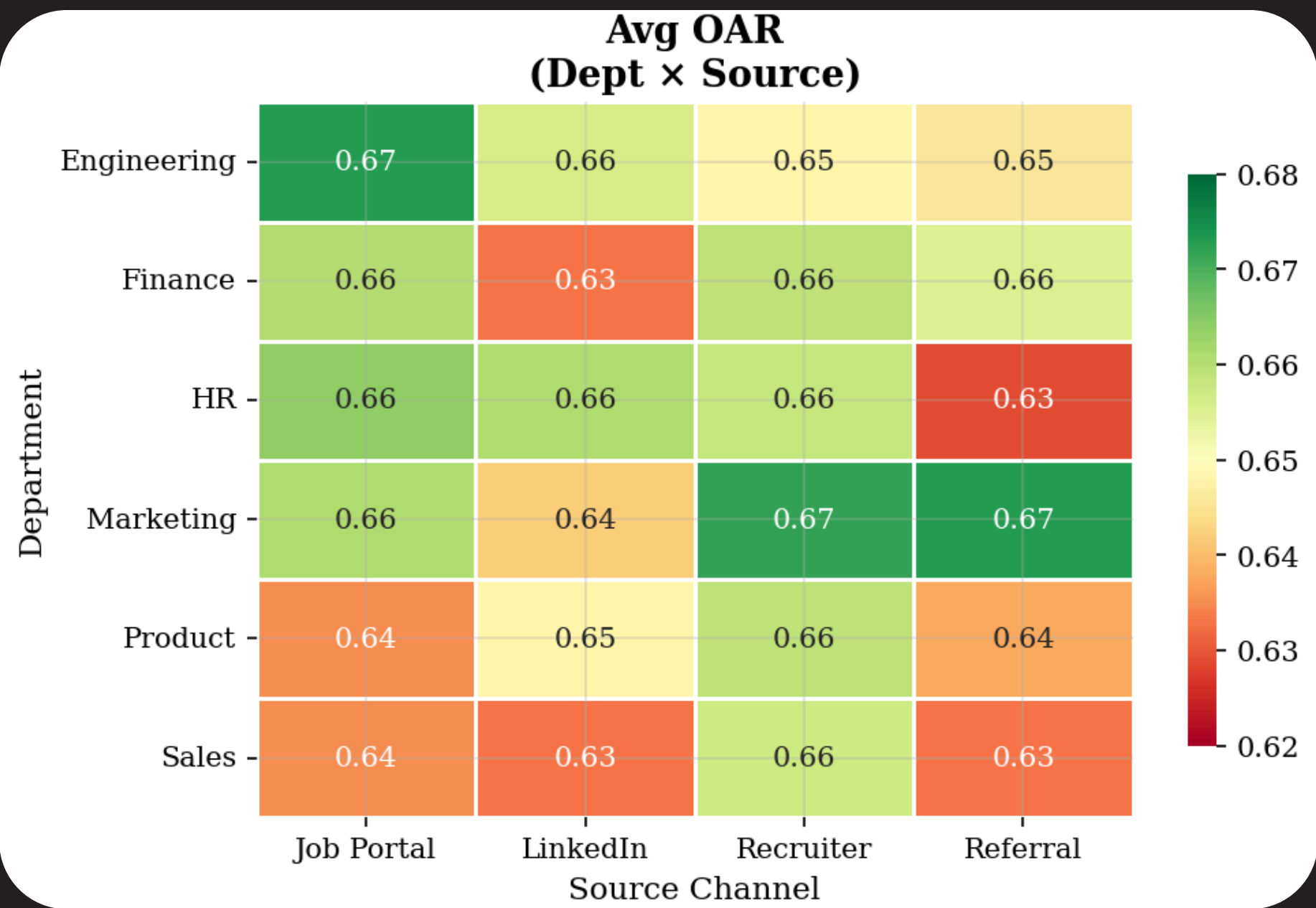


Source channel trade-off

Each channel shows a different efficiency trade-off. **No single channel wins on all three metrics.**

Job Portal — fastest (46.5 days)
LinkedIn — lowest cost (\$5,148)
Recruiter — highest OAR (66%)

Match the channel to the role



No single source works best for all roles

OAR varies by department and source channel.

Aligning the right channel to each department is key to improving acceptance rates.

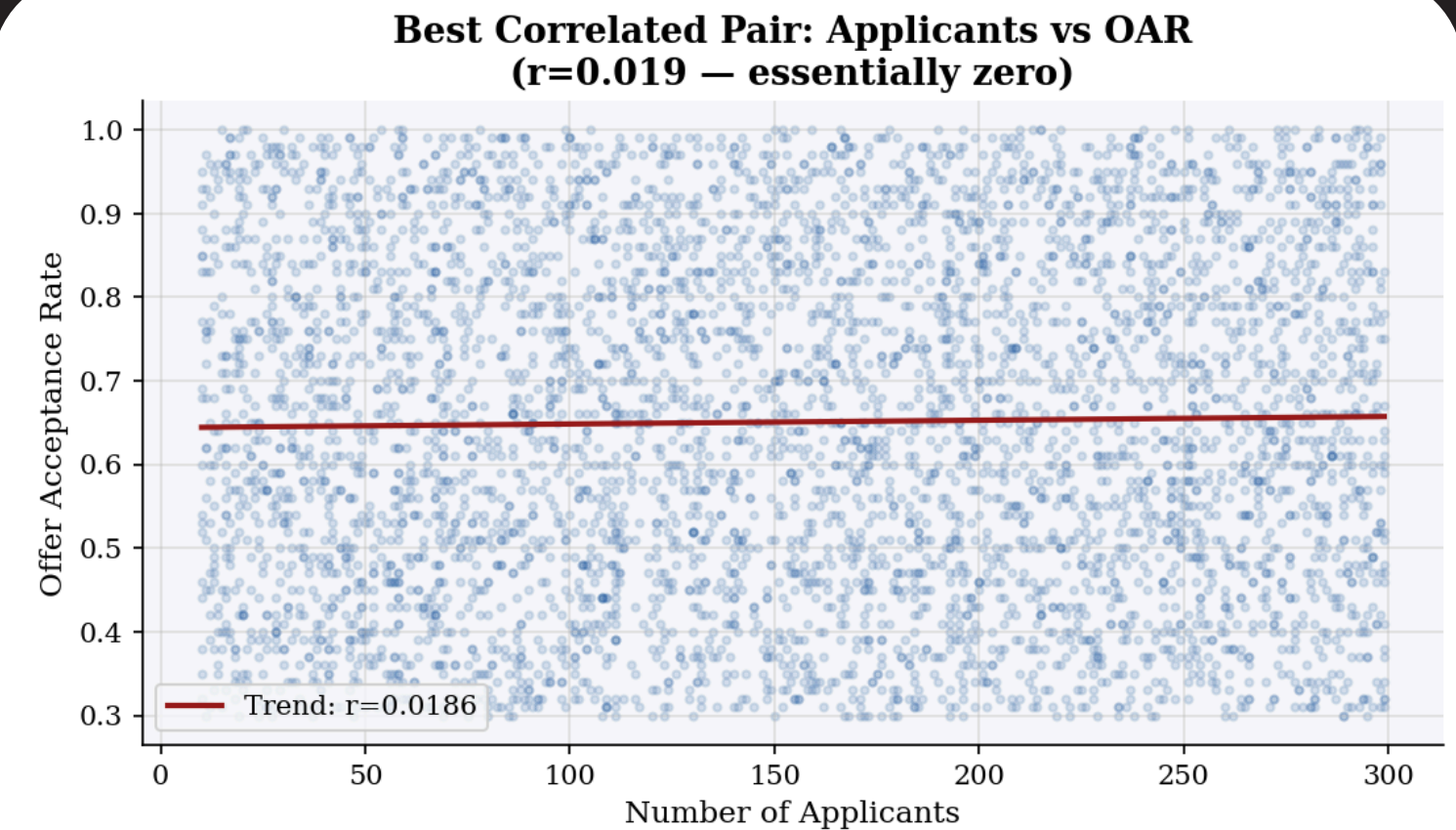
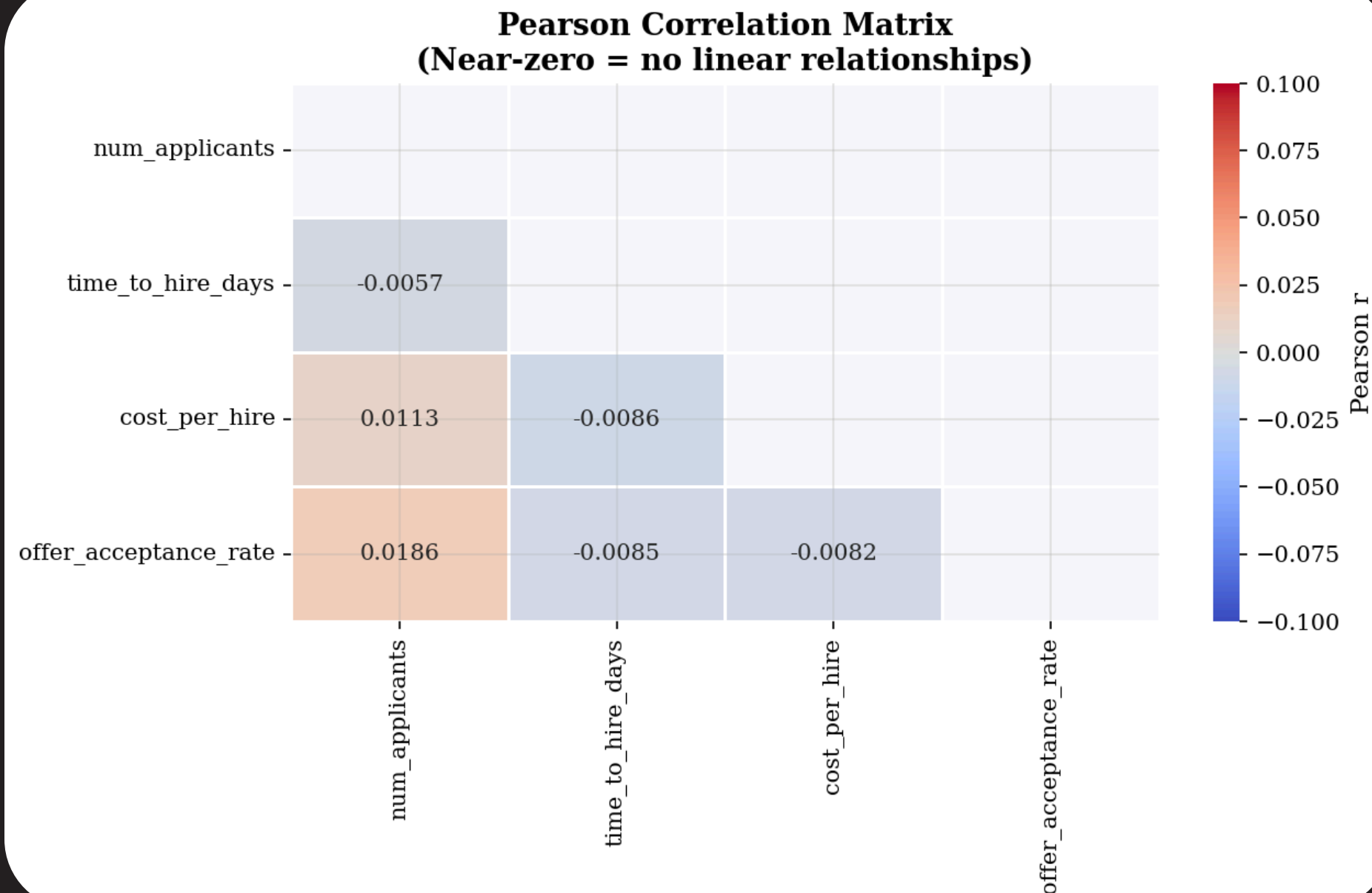
Best channel per department

Engineering	Job Portal (0.67)
Finance	Recruiter / Referral (0.66)
HR	Job Portal / LinkedIn / Recruiter (0.66)
Marketing	Recruiter / Referral (0.67)
Product	Recruiter (0.66)
Sales	Recruiter (0.66)

Key takeaway

Recruitment channel effectiveness differs across departments. **Channels that work in Engineering may not perform equally in Product or Sales** — indicating the need for a targeted, department-specific sourcing strategy to improve OAR while reducing unnecessary cost and hiring time.

Numbers don't lie — but they don't always correlate



Increasing candidate volume does not improve OAR
Even the strongest correlation in this dataset ($r = 0.019$) is **statistically negligible** — the trend line is nearly flat.

No strong linear drivers of offer acceptance identified

Recruitment performance is influenced more by **process quality, candidate fit, and sourcing effectiveness** than by simply increasing hiring activity or candidate volume. This confirms the need for a **predictive, data-driven model** rather than relying on volume-based strategies.

Turning weak signals into strong predictor

Efficiency ratio

- **cost_per_applicant**
→ $\text{cost_per_hire} / \text{num_applicants}$
- **cost_per_day**
→ $\text{cost_per_hire} / \text{time_to_hire}$
- **applicants_per_day**
→ $\text{num_applicants} / \text{time_to_hire_days}$

Role context

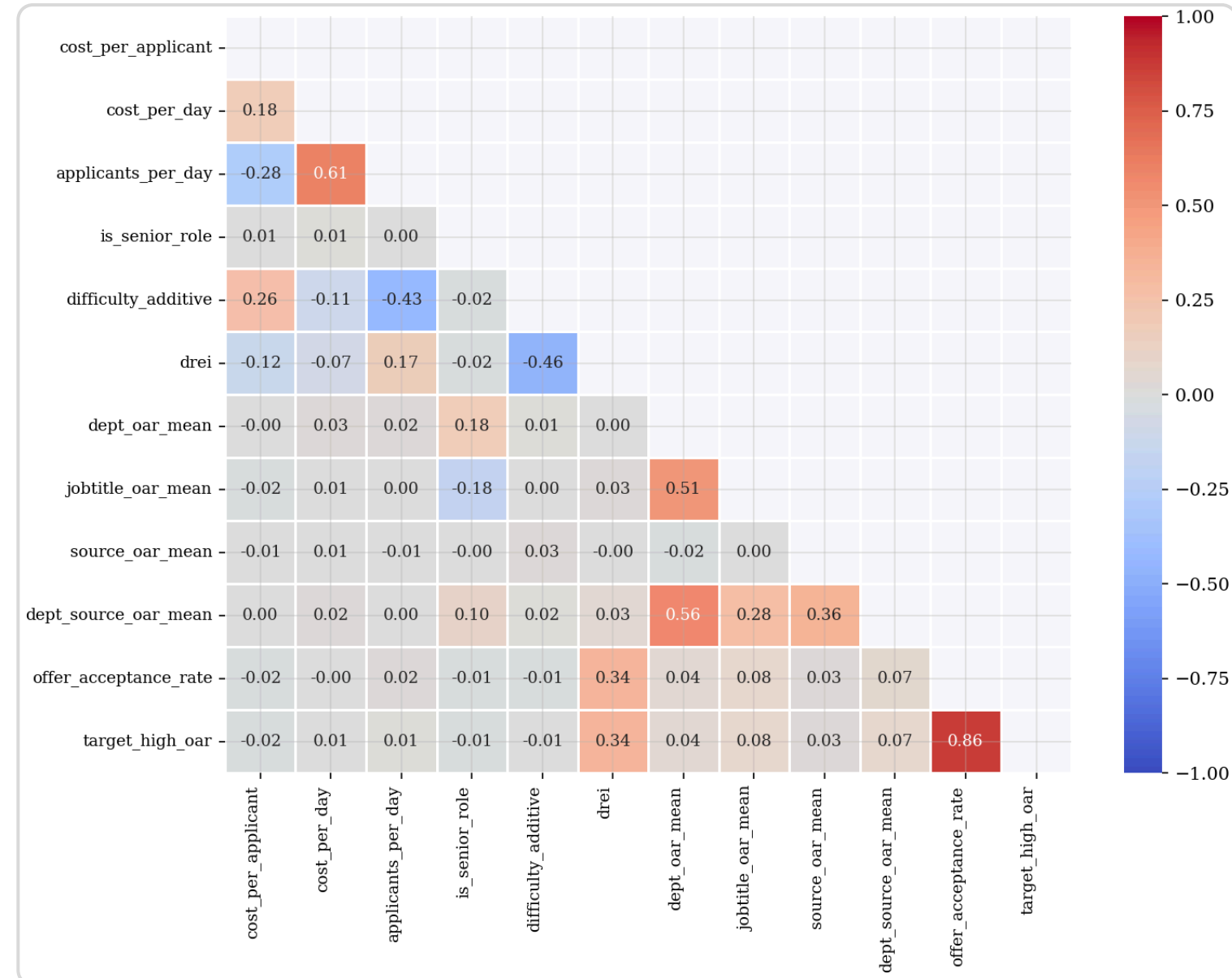
- **is_senior_role**
→ keyword flag (strategist, manager, specialist, executive)

Difficulty additive

- **difficulty_additive**
→ $(\text{zscore}(\text{cost_per_hire}) + \text{zscore}(\text{time_to_hire})) / 2$

Target encoding

- **dept_oar_mean**
- **jobtitle_oar_mean**
- **source_oar_mean**
- **dept_source_oar_mean**



Engineered features reveal better signals

Raw features showed near-zero correlations. After engineering, **target-encoded features (dept, job title, source, dept×source OAR means) capture meaningful patterns** — providing the model with stronger predictive signals than raw inputs alone.

Preprocessing: scaling & balancing the data

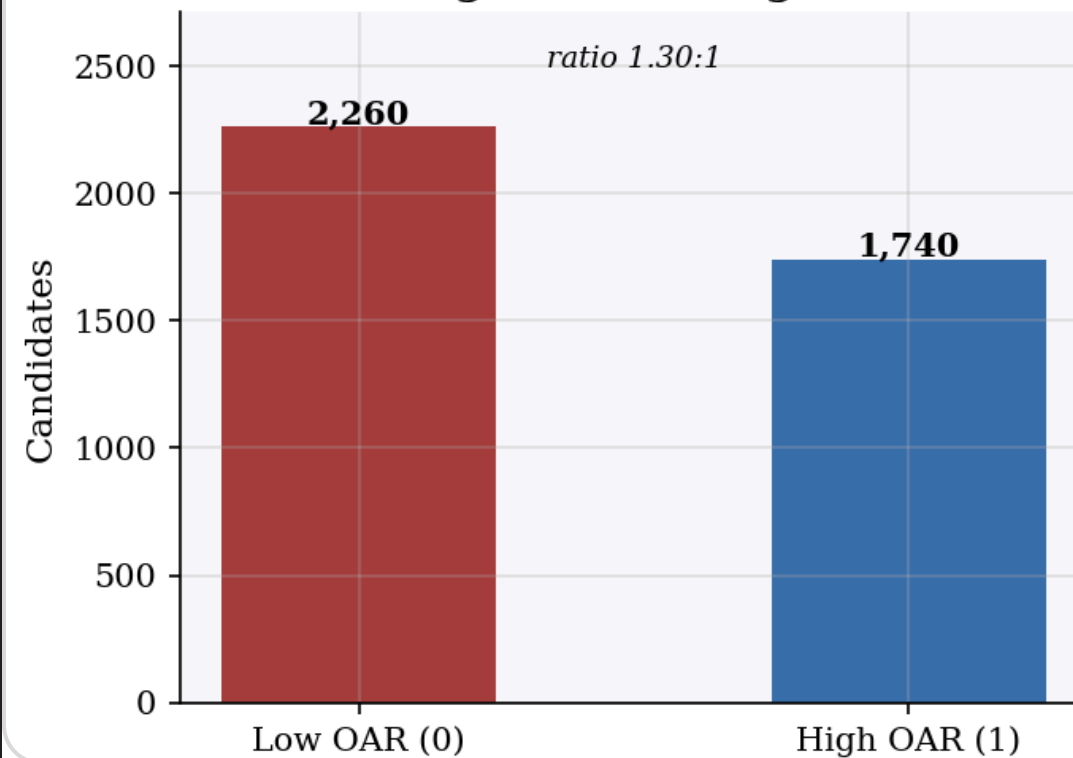
Standard scaling

Applied for distance and gradient-based models — **LR, KNN, SVM** — to ensure features are on a comparable scale and improve training stability. Raw (unscaled) data preserved for **tree-based models (XGBoost)** which are insensitive to scaling.

SMOTE — Synthetic Minority Oversampling

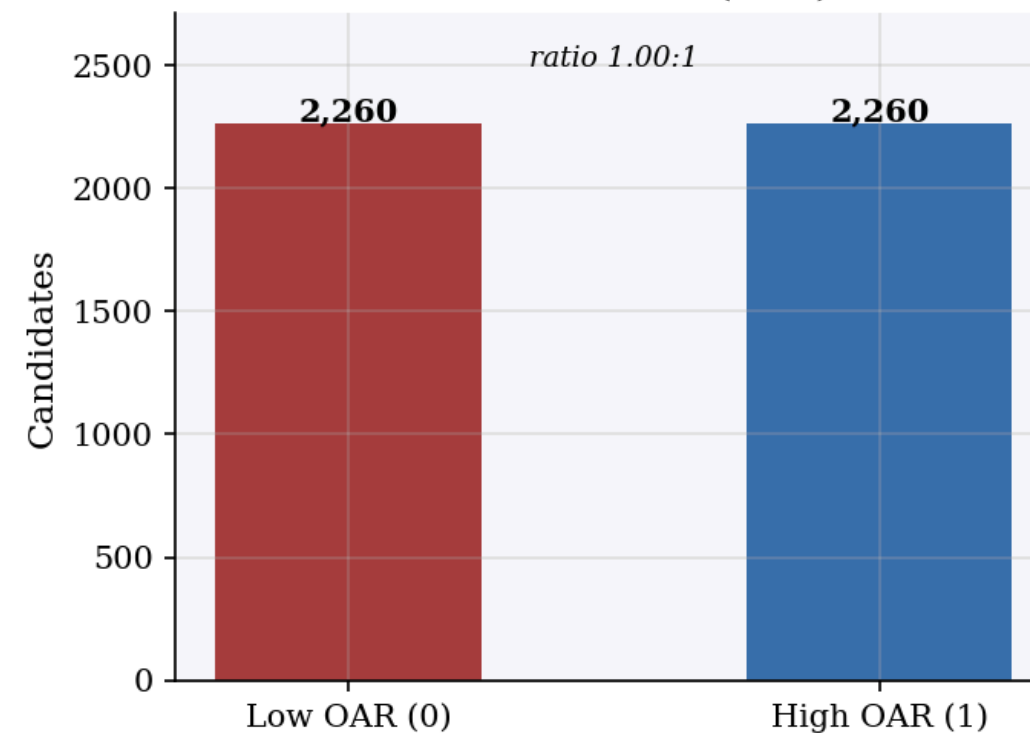
Applied **exclusively to the training set** to address class imbalance by generating synthetic samples for the minority class. Test set kept **unchanged** to maintain fair model evaluation.

Original Training Set



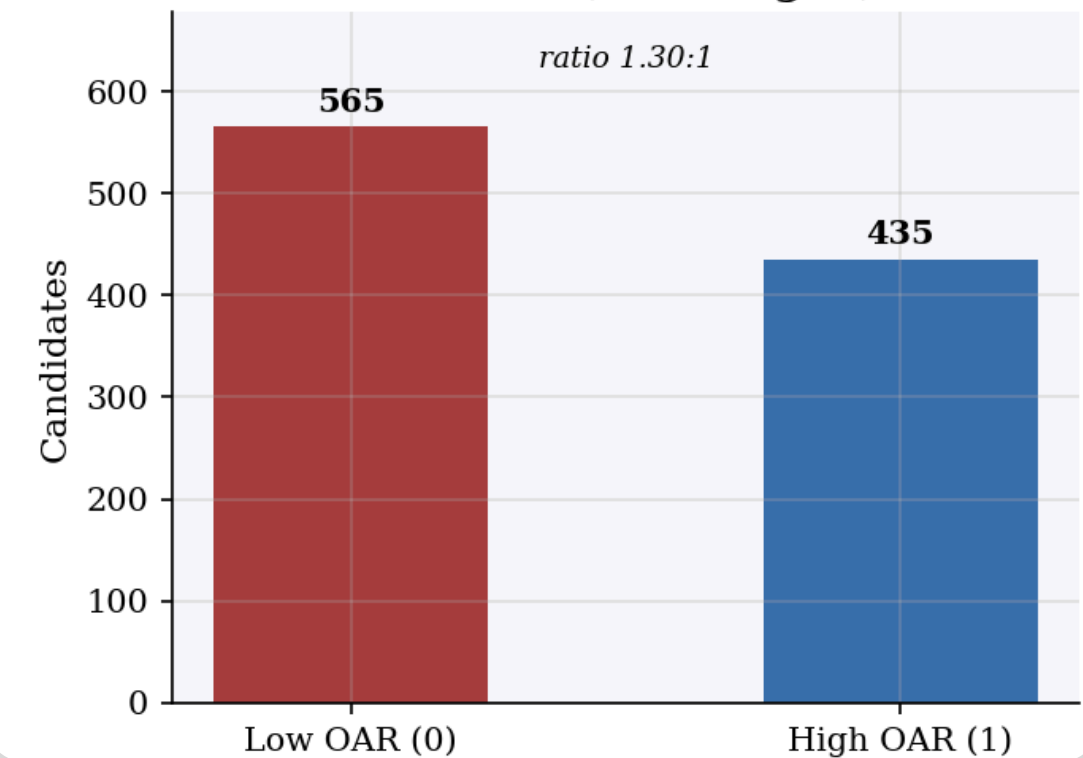
Imbalanced

After SMOTE (raw)



Balanced

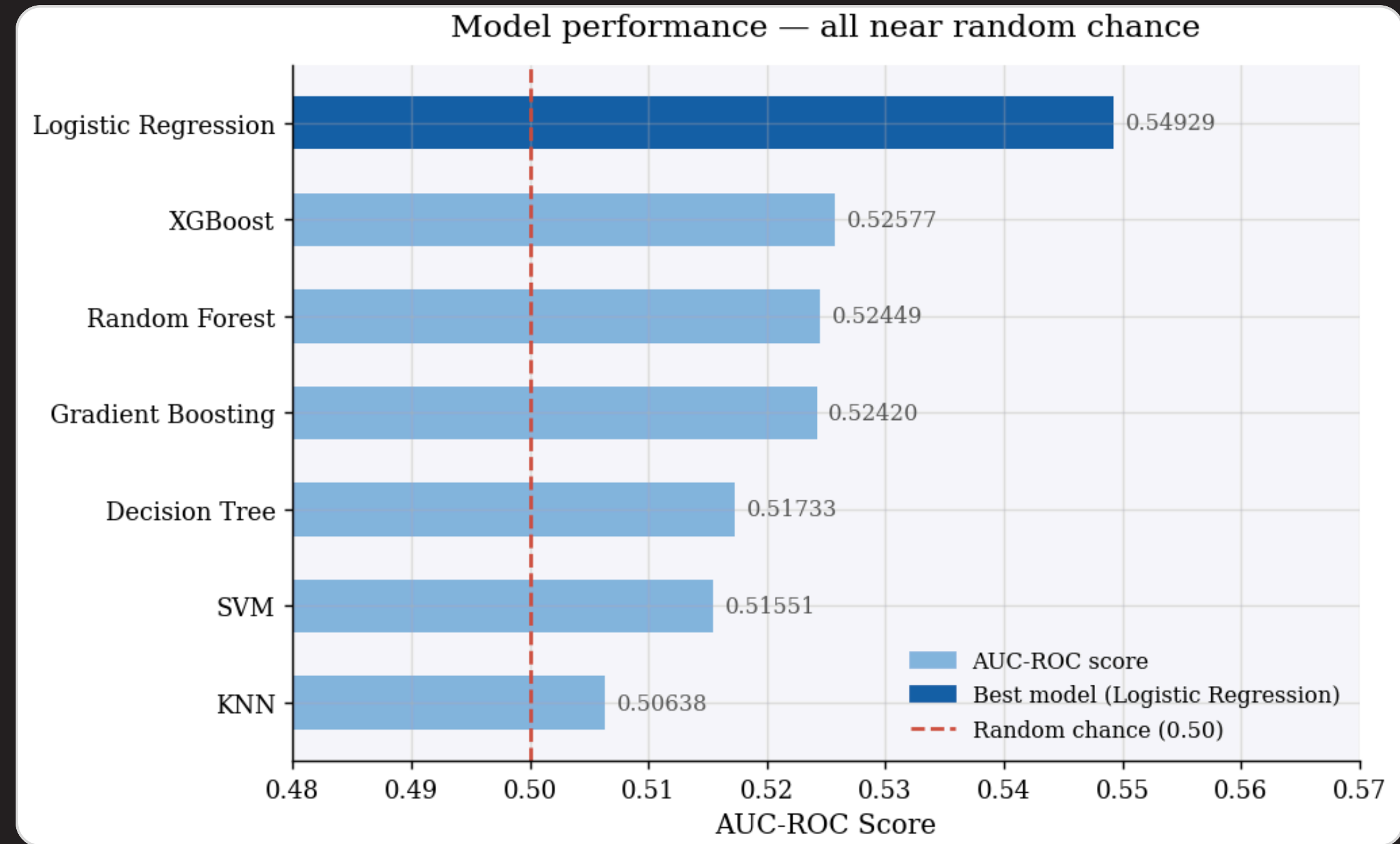
Test Set (unchanged)



Fair evaluation

Our initial models are guessing — not learning

Every model scored close to 0.5 — **no better than a coin flip**. The data we gave them lacks a critical signal: context within each department.



Why are models failing?

All scores cluster between **0.506 – 0.549**, barely above random. The models lack **department-level context** — they can't distinguish a high performer from a low one within the same team.



New Feature: DREI

Department-Relative Efficiency Indicator

— a yes/no flag: was this record outstanding compared to its own department?

- ✓ Lower cost than department median
- ✓ Faster hire than department median
- ✓ Higher OAR than department median

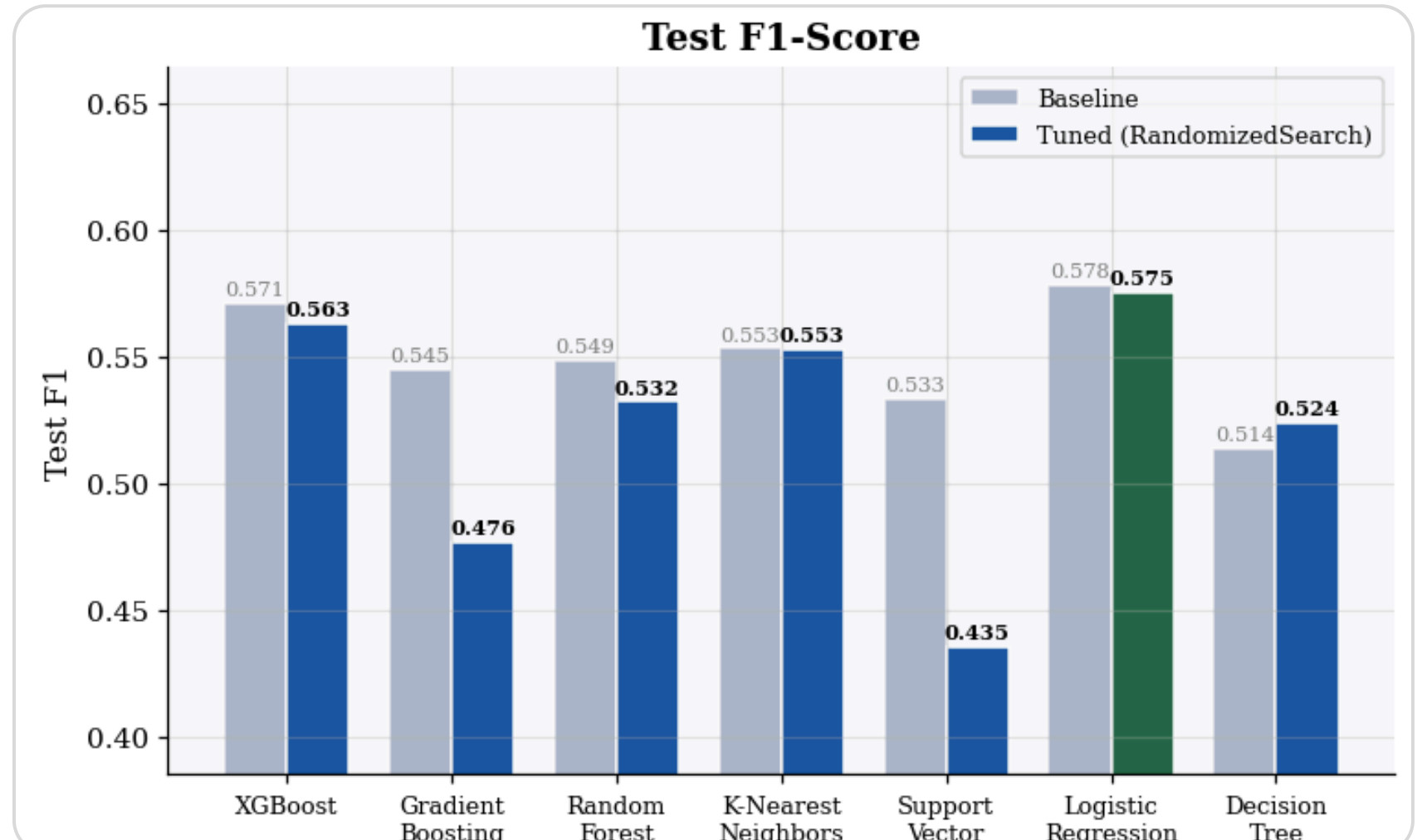
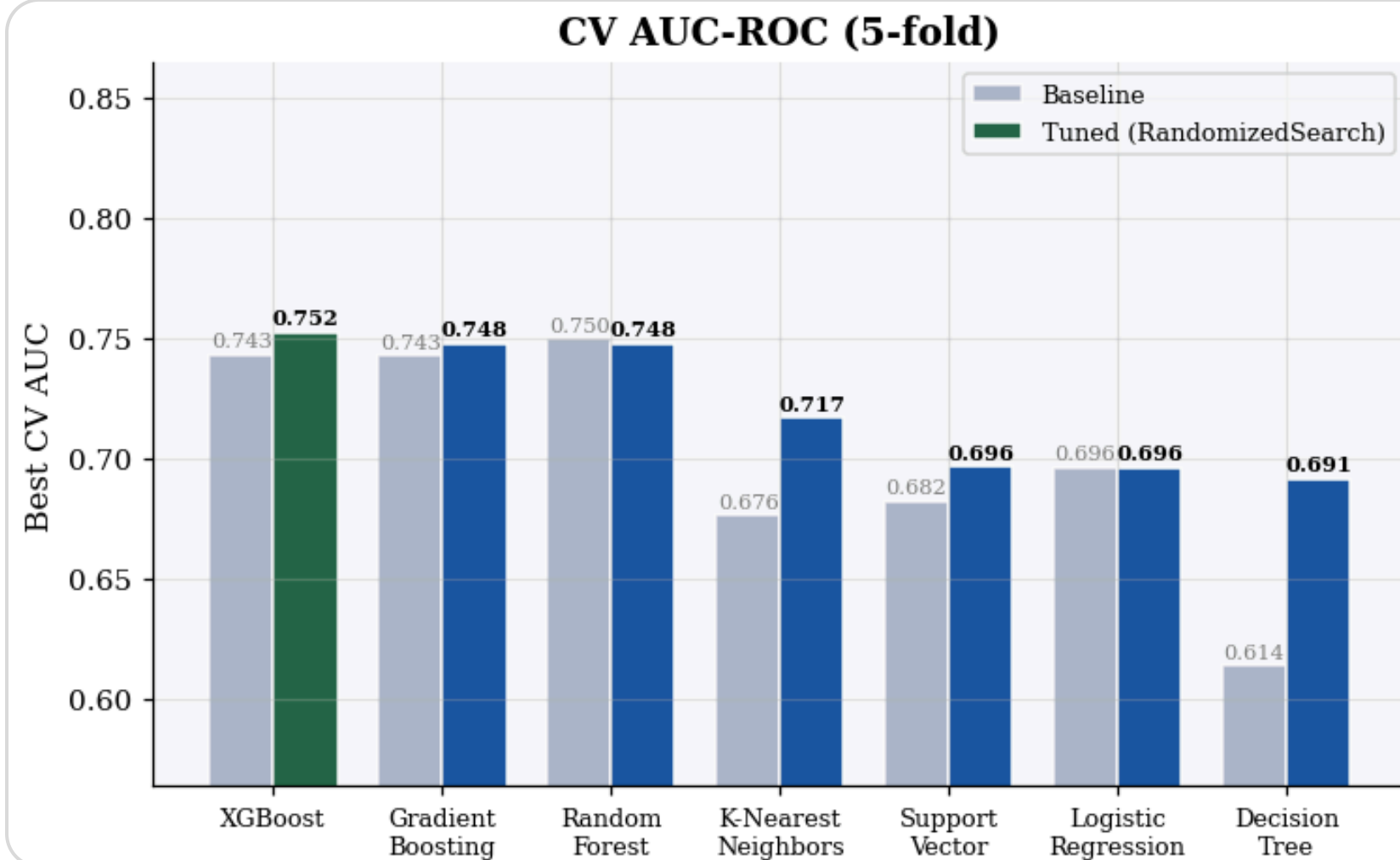
DREI = 1 when all three conditions are met.

Which model should we deploy?

Comparing all seven models before and after tuning — **AUC** measures risk ranking ability, **F1** measures prediction accuracy.

❶ **AUC:** XGBoost leads at **0.752** after tuning, with Gradient Boosting and Random Forest close behind at 0.748. Margins are tight at the top.

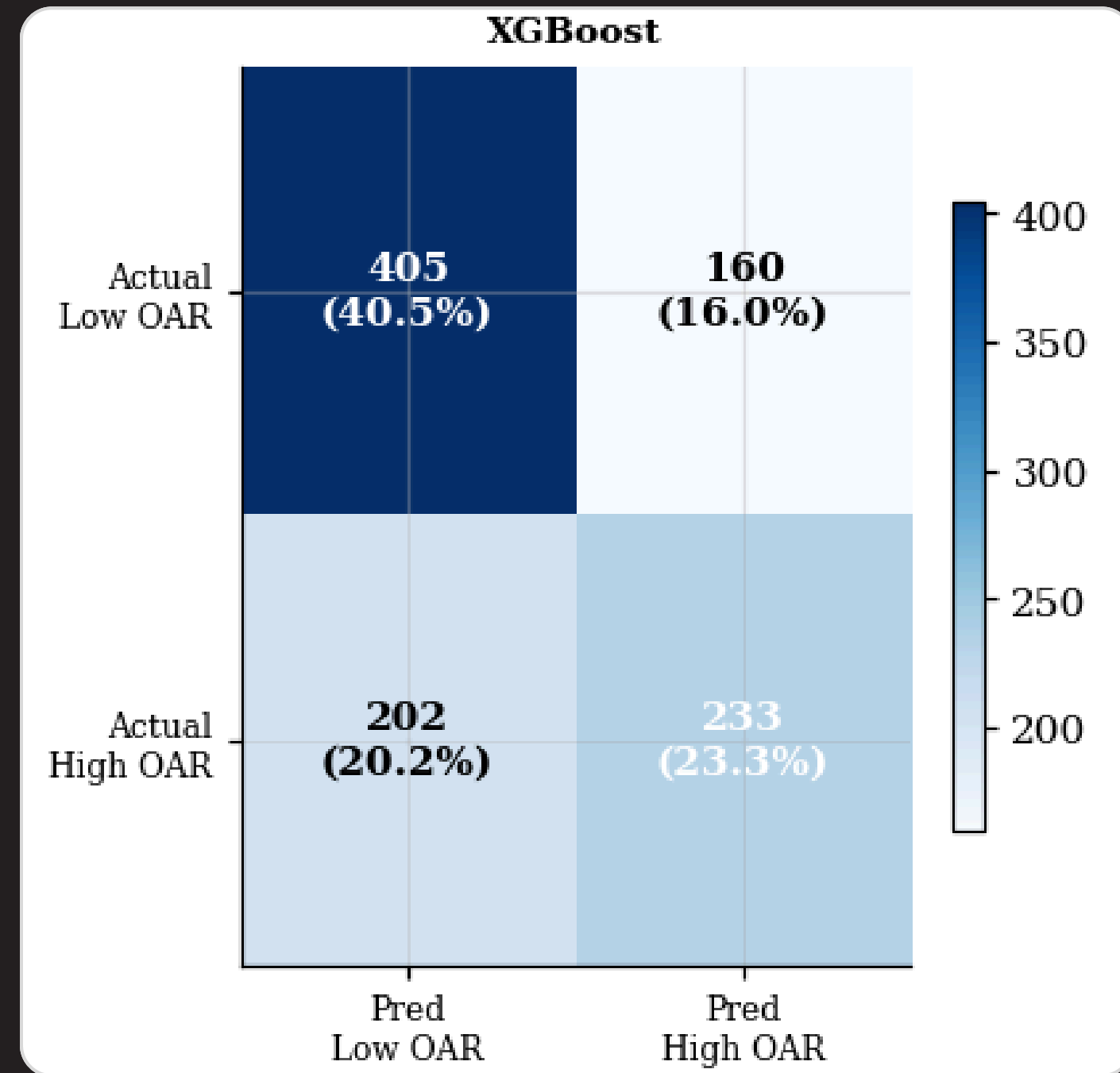
❷ **F1:** Logistic Regression takes the lead at **0.575**. Three models dropped after tuning (GB, RF, SVM) — a warning sign of **overfitting**.



No single model wins on every metric — which is why we evaluate holistically. **XGBoost** offers the strongest overall balance between AUC-ROC and F1, making it the recommended model for deployment.

What the model actually tells HR

Diving into the confusion matrix reveals why XGBoost is our recommended model — it achieves the best trade-off between catching true acceptances and minimizing false alarms.



TN — 405

Recruiter time & cost saved

These candidates were correctly flagged as likely to decline. **HR can deprioritize them early**, saving interview slots, offer prep, and negotiation effort.

✓ TP — 233

Offers worth fast-tracking

Correctly predicted acceptances. **HR can prioritize these candidates**, allocate more attention, and close faster — directly improving OAR.

⚠ FP — 160

Wasted hiring effort

Offers extended to candidates expected to accept — but they declined. Each FP = **recruiter time + interview costs + delayed hiring cycle**.

✗ FN — 202

Missed good hires

Candidates who would have accepted but were deprioritized. **These are lost hires** — often the costliest error in recruitment.

Why the model predicts what it predicts

SHAP analysis explains XGBoost's predictions — confirming it's explainable, trustworthy, and aligned with recruitment logic.

Figure 5.8: SHAP Summary Plot — XGBoost (Test Set, n=1,000)

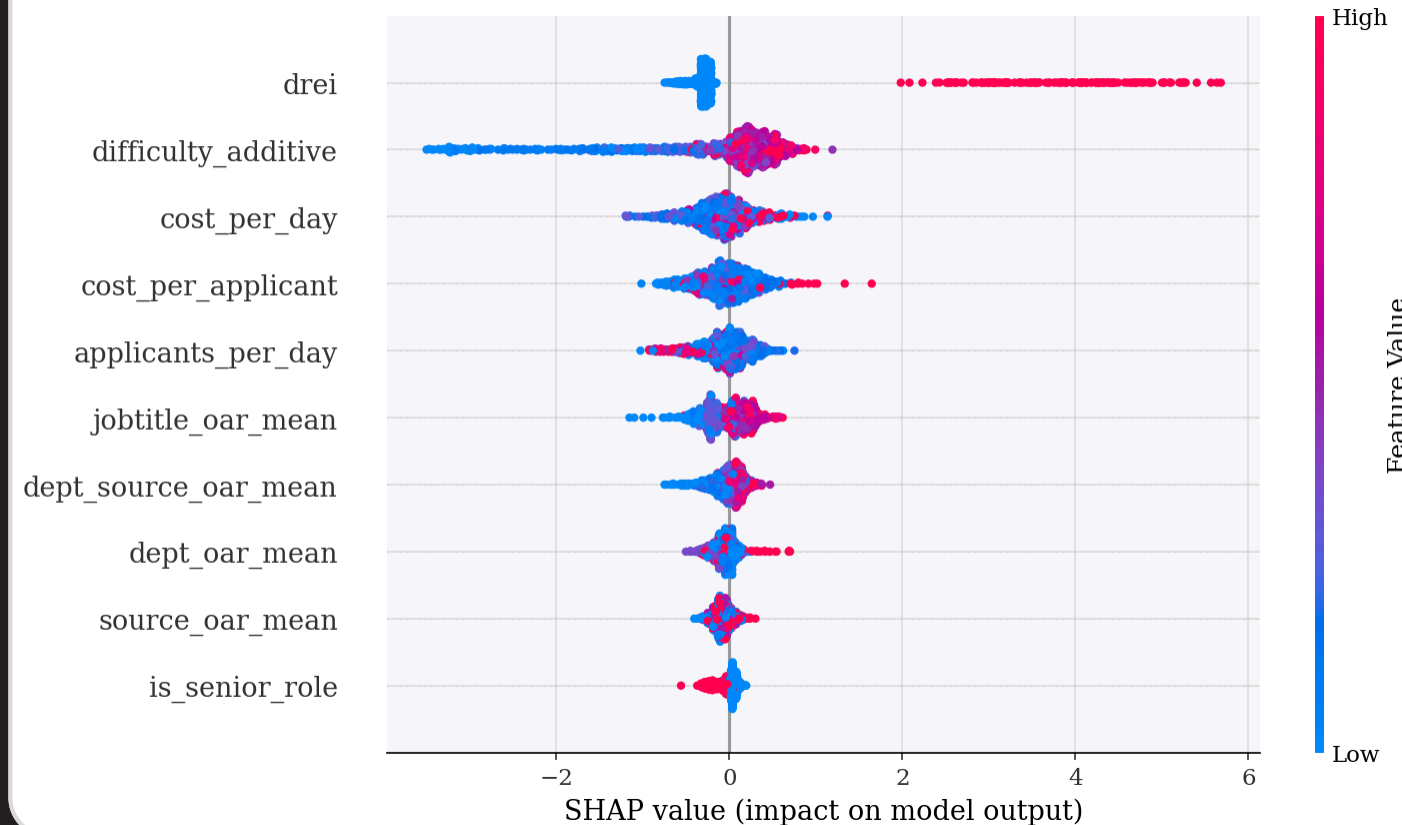


Figure 5.11: SHAP Waterfall — High OAR Prediction Breakdown

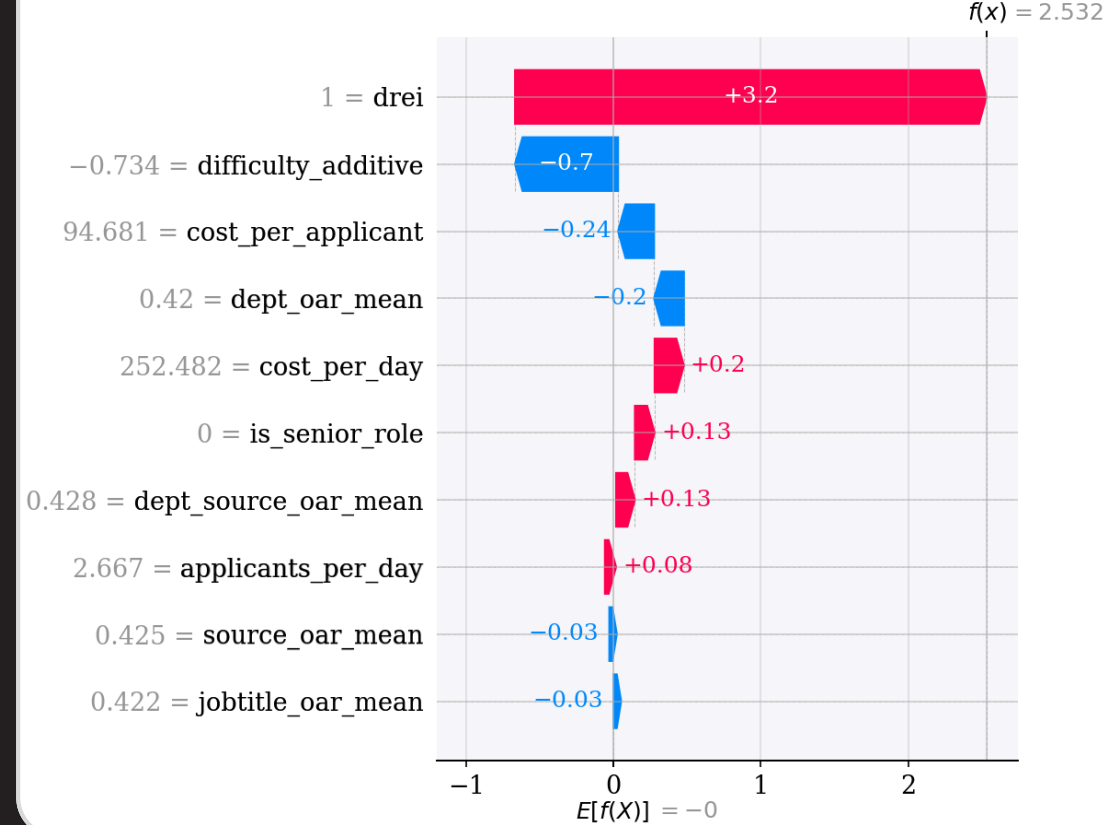
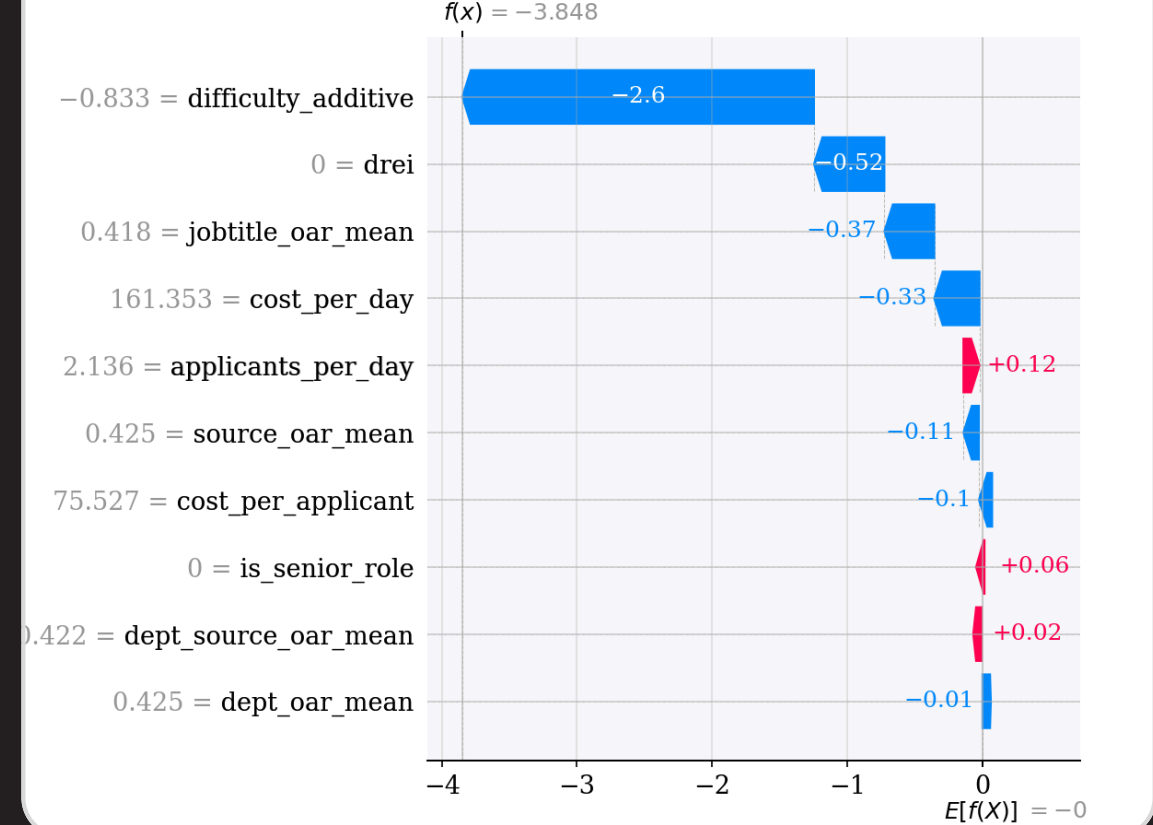


Figure 5.11: SHAP Waterfall — Low OAR Prediction Breakdown



DREI is the #1 driver

When a candidate outperforms their department on cost, speed, and quality — **the model is highly confident they will accept**. HR should prioritize outreach, reduce delays, and allocate senior recruiter attention.

High difficulty = High rejection risk

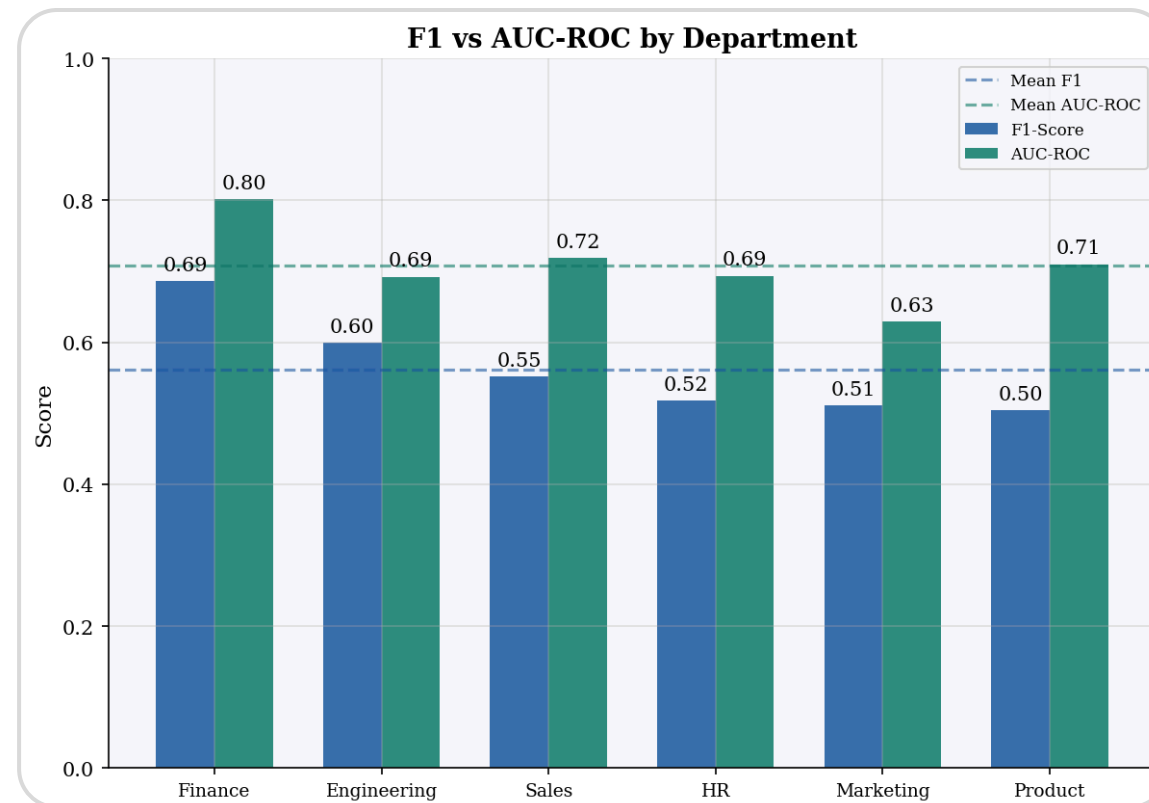
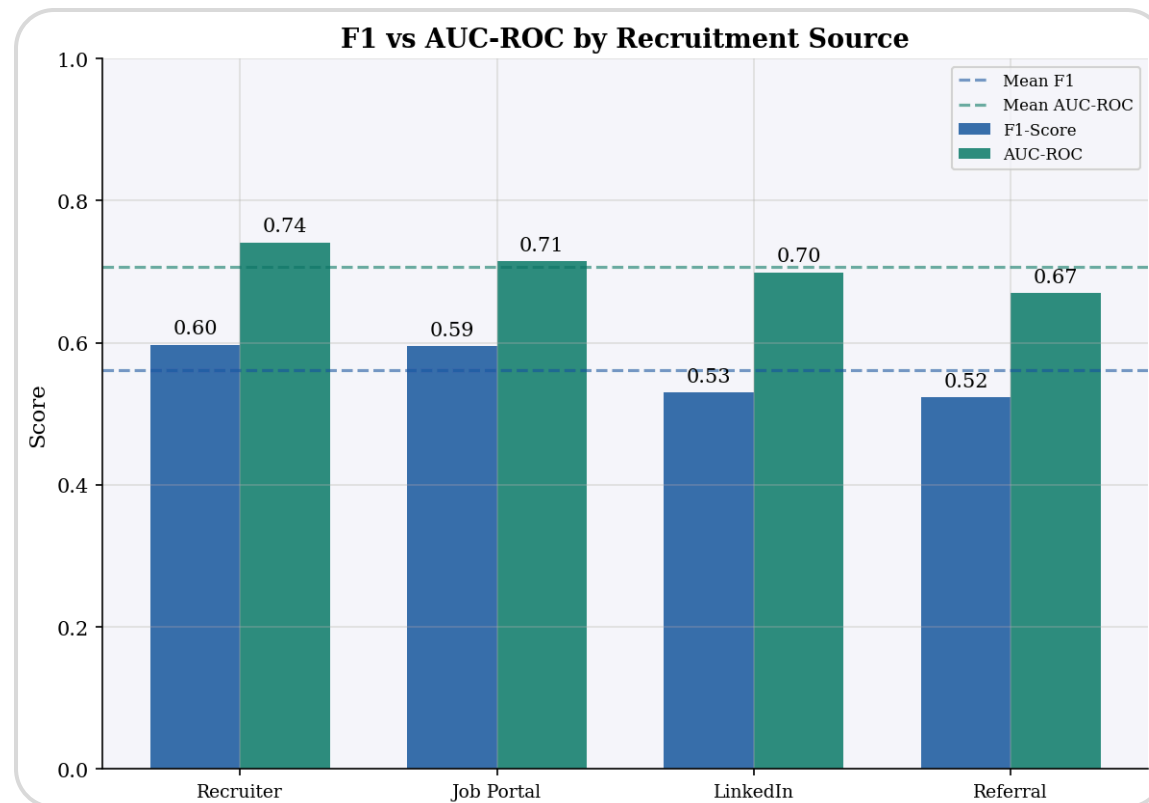
Roles that are both expensive and slow to fill signal a **poor candidate-role fit or sourcing mismatch**. HR should review the job description, switch channels, or adjust compensation before extending offers.

Cost efficiency signals candidate quality

High cost-per-day and cost-per-applicant **negatively impact the prediction**. Efficient hiring processes — fewer applicants, faster closes — correlate with candidates more likely to accept.

Is the model fair across all groups?

A fair model should perform consistently across subgroups. We tested XGBoost across recruitment sources and departments to identify where predictions may be biased or unreliable.



Finance is the model's sweet spot

Highest AUC (0.80) + strong F1 (0.69)
Finance candidates are the most predictable. **HR can trust model predictions most confidently here** — prioritizing Finance pipeline management based on model scores will yield the best ROI.

AUC is consistent

Model ranks fairly across all groups
AUC-ROC stays between **0.67–0.80** across all sources and departments — meaning the model's **ranking ability is reliable** regardless of which group a candidate comes from.

F1 varies — bias risk

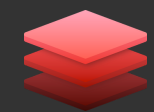
Product, Marketing, HR, Referral, & LinkedIn underperform
F1 drops to 0.50 (Product) and 0.52 (Referral) — near random. HR relying on predictions for these groups may make more wrong decisions, creating unfair candidate treatment.

Fairness verdict

The model performs well overall but shows **uneven F1 across departments and sources**. For groups like Product, Marketing, and Referral hires, HR should **combine model scores with human judgment** rather than relying on predictions alone — reducing the risk of systematically disadvantaging certain candidate groups.

Beyond the model — a dashboard HR can use today

The project doesn't stop at finding the best model. We built a ready-to-use recruitment decision dashboard powered by Streamlit, integrated with GitHub for easy access and deployment.



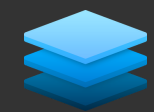
Streamlit Frontend — Interactive Dashboard

Open-source Python framework for building data apps instantly. No web development experience needed — HR teams can interact with predictions through a clean, browser-based UI.



GitHub Version Control & Deployment

The dashboard is hosted directly via GitHub integration. Any model update is pushed to the repo and reflected in the live app — **no manual deployment needed**.



XGBoost Model Backend — Prediction Engine

The trained model runs in the background. HR inputs candidate data, and the model returns an **acceptance probability score in real time**.



1

Candidate Input Form

HR enters candidate details — department, source, cost, time-to-hire — and gets an instant **offer acceptance probability score**.

2

High / Low OAR Classification

The model classifies each candidate as **High OAR ($\geq 70\%$)** or **Low OAR ($< 70\%$)** with a confidence level, helping HR prioritize who to pursue.

3

Recruiter Action Recommendation

Based on the score, the dashboard suggests a clear next step — fast-track, review, or deprioritize — turning model output into recruiter actions.

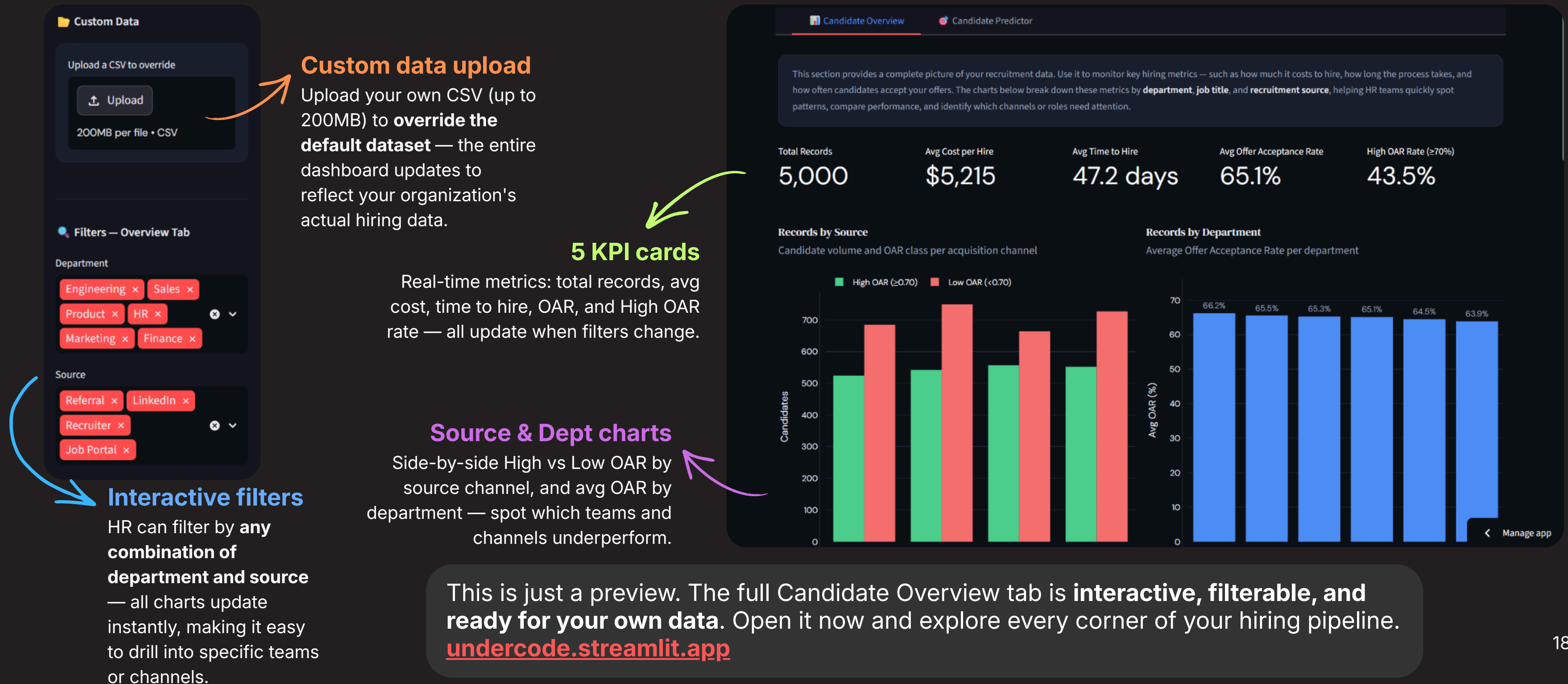
4

Always Up-to-Date via GitHub

Model updates, bug fixes, and new features are pushed to GitHub and **automatically reflected in the live dashboard** — zero manual redeployment.

RePort — Candidate Overview Tab

The first tab gives HR teams a complete, interactive view of their recruitment pipeline — filterable by department and source in real time.



RePort — Candidate Predictor Tab

Fill in candidate details and get an instant offer acceptance prediction — single candidate or entire batch. The model explains every score so HR knows exactly what to do next.

Single predictor

Input one candidate at a time with **full control over every parameter** — ideal for live decisions during interviews.

Click "Predict Outcome"

XGBoost runs instantly in the background — **no waiting, no loading screen.**

This section uses a machine learning model to predict whether a recruitment process is likely to result in a **High OAR** (≥ 0.70) or **Low OAR** (< 0.70). Simply fill in the details for a candidate or role — such as department, source, number of applicants, time to hire, and expected cost — and the model will instantly estimate the likelihood of the candidate accepting the offer. You can also upload a CSV file to run predictions for multiple candidates at once.

☒ Single Candidate ☐ Batch Upload

Single Candidate Predictor
Enter recruitment details to predict offer acceptance outcome

Department: Engineering

Job Title: Software Engineer

Source: Referral

Number of Applicants: 150

Time to Hire (days): 30

Cost per Hire (\$): 5000

Expected Offer Acceptance Rate

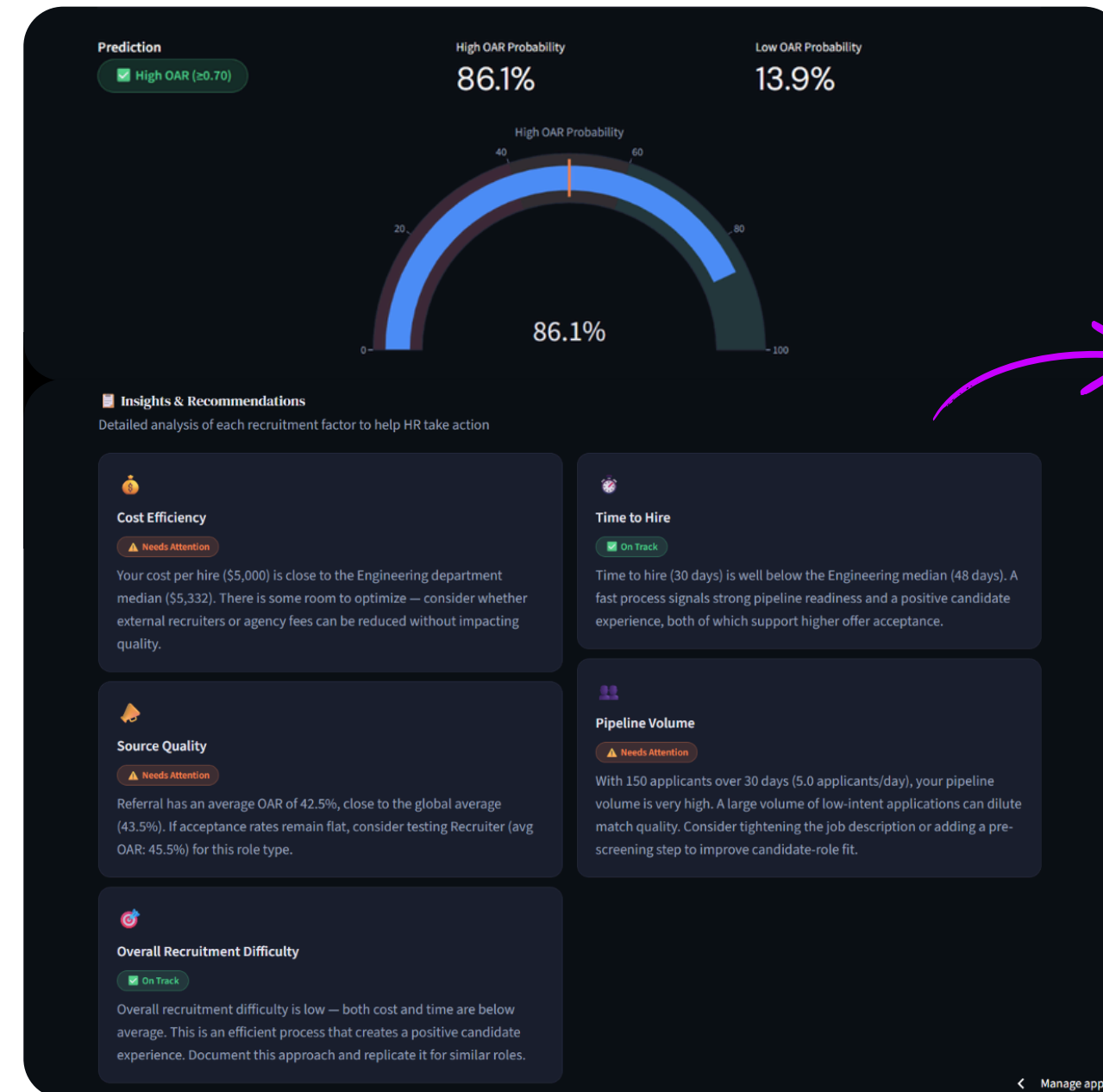
Expected OAR: 0.70

Batch upload

Upload a CSV of your entire pipeline and **get predictions for all candidates at once** — perfect for weekly pipeline reviews.

Fill in candidate details

Select department, job title, and source. Enter number of applicants, time to hire, cost, and expected OAR.



Get a prediction + explanation

The model doesn't stop at 'yes' or 'no.' It shows you **the confidence behind every prediction** and **breaks down exactly which recruitment factors made the difference** — so your next move is always clear.

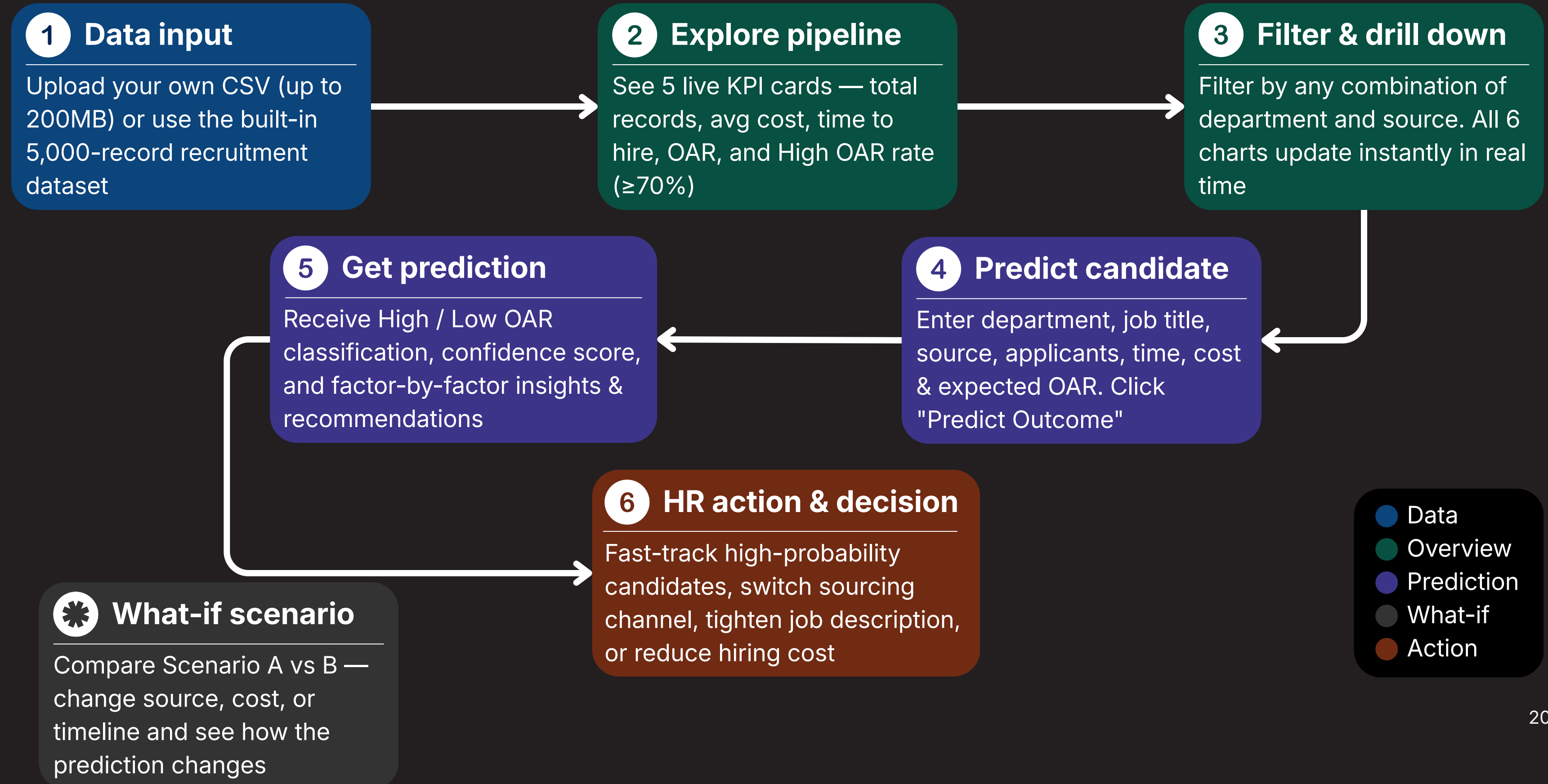
What-If Scenario Comparison

Compare two recruitment approaches for the same role to find the more effective strategy

"What if you changed the source from Referral to Job Portal?"

What if you reduced the cost by \$2,000? RePort answers these questions instantly — open it now and run your own what-if in under 30 seconds." undercode.streamlit.app

RePort — dashboard usage flow



Post-deployment — monitoring & maintenance

Deployment is not the finish line. RePort follows a structured monitoring and maintenance plan to ensure the XGBoost model stays accurate, fair, and reliable over time.

Monitoring schedule

Weekly	Review prediction logs and flag outliers
Monthly	Compare predicted vs actual OAR outcomes
Quarterly	Full model performance review (AUC-ROC, F1)

Retraining process

- 1 Collect & validate new data**
No missing values in key columns
- 2 Re-run stage 2 & 3 notebooks**
Feature engineering, SMOTE, hyperparameter tuning
- 3 Evaluate vs current model**
Must outperform current AUC-ROC (0.71) to replace
- 4 Save & deploy new model**
Push .joblib to GitHub → auto-deploys on Streamlit
- 5 Validate dashboard output**
Run sample predictions and verify results

The cost of guessing vs the value of predicting

Simulated on **2,000** recruitment records comparing the full pipeline against only pursuing candidates predicted as **High OAR (n=736)**. The numbers speak for themselves. Recruiters spend less time on low-probability candidates and more on building relationships with likely acceptors.



Candidates Focused on

736

Out of **2,000** total
only **36.8%** needed for same outcome



Cost Saved

\$6.9M

From **\$10.82M** → **\$3.90M**
saved on non-converting candidates



Faster time-to-fill

-2.48 days

Reduce **time-to-hire**,
eliminate vacancy costs



OAR Improvement

+0.05

From **0.66** → **0.71**
crosses the **0.70** threshold

Decision quality improved by Data-driven

HR moves from **gut-feel to model-backed decisions**, every candidate comes with a probability score, factor breakdown, and a recommended action.

Strategic recommendations for smarter hiring

Immediate actions

- **Deploy RePort to HR teams**
Use prediction scores to prioritize candidates starting with the next open role.
- **Align sourcing to High OAR channels**
Recruiter & Job Portal outperform others. Shift budget by department using the heatmap.
- **Run what-if before every offer**
Compare two scenarios in RePort. Pick the higher predicted OAR. Takes under 30 seconds.
- **Reduce pipeline volume — focus on fit**
More applicants ≠ higher OAR. Tighten job descriptions, add pre-screening steps.
- **Track predicted vs actual OAR monthly**
Log outcomes every month this is how you know when to retrain the model.

Long-term strategy

- **Expand training data beyond 5,000 records**
More data means better generalization and higher AUC. Retrain quarterly as new recruitment data comes in.
- **Enrich candidate data collection**
Current model relies on process metrics only (cost, time, source). Adding interview scores and assessment results shifts the model from predicting process efficiency to predicting candidate role fit a stronger signal.
- **Extend model coverage**
Current hyperparameter tuning was constrained by time and resources. Broader tuning in future iterations can push model performance closer to the $\geq 80\%$ target.
- **Integrate real-time HR data feed**
Connect RePort directly to your HR system. Live data means live predictions no manual uploads, no stale inputs.



Hire Smarter Not Harder

A data-driven approach to optimizing recruitment efficiency and predicting offer acceptance

